002

Execution modes and Syscalls

Operating Systems

Execution modes and Syscalls

1) Executive Summary

Phase 2 advances the current bare-metal multiprogramming kernel from "preemptive task switching works" to "preemptive switching works under protected kernel control."
The phase introduces a strict user/kernel execution boundary, a minimal syscall ABI, and fault-driven isolation behavior, while preserving the existing timer-driven round-robin context-switch core already built in Phase 1.

The expected outcome is a kernel that can:

- Keep current multitasking behavior stable.
- Run user tasks in unprivileged mode.
- Accept controlled requests through syscalls.
- Contain user-space faults without collapsing the whole system.

2) Confirmed Phase 1 Baseline (Input to this Phase)

- DMTimer2 interrupt path is implemented.
- IRQ-driven preemptive round-robin scheduling is implemented.
- Context save/restore and PCB-based switching are implemented.
- Multiple user programs execute and interleave over time.

Phase 2 must treat these as working baseline capabilities, not re-implementation targets.

3) Kernel and User Mode

3.1 Objectives

- **Privilege correctness**: kernel-only work runs in privileged processor modes; application code runs in ARM **USR** (or the unprivileged mode your boot contract defines). User code must not execute privileged operations except through the intended hardware mechanisms below.
- **Correct save/restore at every USR ↔ kernel crossing**: every return to **USR** restores the **selected task's** program-visible registers, stack pointer, link register where applicable, and CPSR/mode bits for **USR**.

3.2 Execution Domains

- **Kernel**: board/bootstrap bring-up, **timer IRQ** handler and scheduler, **svc** syscall dispatcher (handler bodies in Section 4), **fault** handlers (policy in Section 5), PCB/accounting.

- **User**: instruction stream and stacks you map for tasks; linker layout and optional MPU/policy define what USR may touch.

3.3 How this document splits ARM behavior

ARMv7-A groups IRQ, Supervisor (svc), prefetch/data aborts, etc. under the **general exception model**. Operationally Phase 2 is easier to validate if naming matches **intent**:

Name in this document	ARM mechanism	Synchronous vs asynchronous
Initial boot transition	First legal entry to **USR** after boot (constructed context + exception-style return)	N/A (one-time handoff)
Interrupt path	**IRQ** (e.g. DMTimer2)	Asynchronous to user instruction stream
Exception path	**Data abort** / **prefetch abort** (and related fault status)	Synchronous to a faulting instruction
Syscall path	**Supervisor call** via **svc**	Synchronous to the **svc** instruction

3.4 Initial boot transition (kernel → first user task)

Purpose: leave reset/bootstrap **privileged** execution and start the first runnable task in **USR** with a valid frame.

- Kernel finishes minimal bring-up (clocks, stacks, interrupt controller, timer as in Phase 1).
- For each task to be run, build a **synthetic first context**: USR **SP**, **PC** (entry symbol), **CPSR** snapshot appropriate for **USR** and instruction set state (ARM/Thumb as you link it).
- Enter the first task using the **same return-to-USR mechanism** you use after later scheduling decisions (not a random jump that leaves wrong mode bits).
- From this point on, DMTimer **IRQ** can preempt **USR** like Phase 1, but with USR-level save/restore.

Trace (required once per cold start)
MODE_SWITCH KERNEL_TO_USER pid=<first> reason=initial_launch

3.5 Interrupt path (timer IRQ, preemptive scheduling)

Purpose: asynchronous preemption while a task runs in **USR**; run the scheduler; resume **some** runnable task in **USR**.

- While **PC** executes in **USR**, the timer asserts **IRQ**; CPU vectors to the **IRQ** handler in a privileged mode.
- Handler records **which task** was interrupted and saves the **USR**-visible state on the shared trap-frame contract (ordering compatible with other paths; see Section 6).

- Timer source is acknowledged/cleared per your device code; kernel runs **round-robin** selection as in Phase 1.
- Handler restores the **next** chosen task's frame and performs an exception return into **USR** at that task's resumption PC.

Traces
MODE_SWITCH USER_TO_KERNEL pid=<n> reason=timer_irq
MODE_SWITCH KERNEL_TO_USER pid=<m> reason=dispatch

(pid=<m> may equal <n> when the RR decision hands the CPU back to the same runnable task.)

3.6 Exception path (faults / aborts)

Purpose: a **USR** instruction causes a **prefetch abort** or **data abort** (illegal address, privilege, alignment—per **IFSR/DFSR** class decoding you implement).

- Fault is taken synchronously relative to execution; privileged **abort-mode** handler runs.
- Classify fault (**IFSR**, **DFSR**, **instruction vs data**) and attach outcome to PCB **per fault policy document** (Section 5): typically terminate or quarantine the faulting task.
- Scheduler picks a **different** runnable **USR** task if one exists.

Traces
MODE_SWITCH USER_TO_KERNEL pid=<n> reason=fault type=<type>
MODE_SWITCH KERNEL_TO_USER pid=<m> reason=fault_recovery

Detailed containment steps remain in Section 5.

3.7 Syscall path (**svc**)

Purpose: deliberate **USR**→kernel requests **yield** / **exit** / **write** through **svc #0** and the register ABI.

- Running in **USR**, user code executes **svc #0** with **r0–r3** loaded per Section 4.
- CPU takes **Supervisor exception** (not IRQ); **SVC** vector runs, saves state on the same family of frame used for coherent return to **USR**.
- Dispatcher validates caller came from **USR**, dispatches by syscall number, runs handler (Section 4).
- Return path sets **r0** result and performs exception return to **USR** for the **task the scheduler left current** (after **yield**, that may not be the syscall's caller).

Traces
MODE_SWITCH USER_TO_KERNEL pid=<n> reason=syscall id=<id>
MODE_SWITCH KERNEL_TO_USER pid=<m> reason=syscall_return id=<id> rc=<rc>

3.8 Unified MODE_SWITCH catalog (all paths)

Every crossing below must be identifiable in serial logs on demand (demo + test runs).

#	Path	Direction	When	Example trace line
1	Initial boot	Kernel → User	First entry to **USR**	MODE_SWITCH KERNEL_TO_USER pid=<first> reason=initial_launch
2	Interrupt	User → Kernel	IRQ taken while **USR** runs	MODE_SWITCH USER_TO_KERNEL pid=<n> reason=timer_irq
3	Interrupt	Kernel → User	After IRQ handler schedules and returns	MODE_SWITCH KERNEL_TO_USER pid=<m> reason=dispatch
4	Syscall	User → Kernel	**svc** from **USR**	MODE_SWITCH USER_TO_KERNEL pid=<n> reason=syscall id=<id>
5	Syscall	Kernel → User	After syscall handler + return	MODE_SWITCH KERNEL_TO_USER pid=<m> reason=syscall_return id=<id> rc=<rc>
6	Exception	User → Kernel	Prefetch/data abort from **USR**	MODE_SWITCH USER_TO_KERNEL pid=<n> reason=fault type=<type>
7	Exception	Kernel → User	After fault policy + reschedule	MODE_SWITCH KERNEL_TO_USER pid=<m> reason=fault_recovery

3.9 Task context shared by all paths

Before any path runs, each task needs a consistent **USR** image: **SP**, **PC**, **r4–r12** initial policy as you define, **LR** if using a C ABI, and **CPSR** for **USR**. **Paths 2–4** only **rotate** among those saved images; they do not replace the need for correct **Path 1** construction for first run per task.

4) System Calls

4.1 What Constitutes a System Call

A **system call** is a **controlled, synchronous transition** from **user mode** into **kernel logic** initiated by user code executing a **svc** instruction with arguments in **r0–r3**. User tasks must obtain kernel services **only** through this path (along with unavoidable exceptions such as faults, Section 5). The kernel verifies that the trap originated from user mode before servicing.

4.2 Execution Path Through the SVC Vector

- User issues **svc #0** with ABI-defined syscall ID in **r0** and arguments in **r1–r3**.
- CPU enters privileged mode at the **SVC** vector entry; trap code saves/restores context compatibly with IRQ preemption semantics.
- **Dispatcher** reads **r0**, validates syscall ID range, invokes the handler for **yield**, **exit**, or **write**, or returns a deterministic error.

- Handler updates PCB/kernel state as specified below.
- **Return path** restores the appropriate user task (possibly not the caller after **yield**) with **r0** holding return value/error per Section 4.8.

Expected behavior: deterministic return codes; no kernel corruption from invalid syscall IDs, invalid arguments, or invalid user pointers.

4.3 Register ABI (ARM / SVC)

- **r0**: syscall ID on entry; return value (int32_t) on exit.
- **r1–r3**: syscall arguments on entry (unused arguments set to zero for stubs that ignore them).
- **svc #0**: canonical trap (**SVCall**).

4.4 Syscall Identifier Table

Symbol	Numeric ID	Purpose (summary)
SYS_YIELD	0	Voluntary reschedule
SYS_EXIT	1	Terminate caller; no return
SYS_WRITE	2	Write bytes (e.g. UART) from user buffer

4.5 SYS_YIELD (Cooperative Scheduling)

User-visible expectation: returns a non-negative value on success; the same logical process may run again later after other tasks share the CPU.

Kernel behavior (ordered):

- Validate trap originated from **user mode**.
- If task still active, mark **runnable** (or equivalent for your PCB).
- Run **scheduler** (Phase 2 keeps Phase 1 round-robin unless you document otherwise).
- Restore and return into **user mode** for the **scheduled** task (may differ from the caller).

4.6 SYS_EXIT (Process Termination)

User-visible expectation: does not return under correct kernel behavior.

Kernel behavior (ordered):

- Record exit code (r1/argument convention as in wrapper below).
- Mark task **terminated**; remove from runnable set.
- Schedule another runnable task; if none, enter a documented idle/halt policy.
- Reclaim resources on a documented cleanup path (stack accounting, PCB flags).

4.7 SYS_WRITE (Privileged Output From User Memory)

User-visible expectation: non-negative returned byte count on success; negative error codes per Section 4.8.

Kernel behavior (ordered):

- Validate **file descriptor** (this phase expects at least **fd == 1**).
- Validate **buf/len** as a readable **user** range inside the tasks' mapped regions **before** dereferencing (len capped to a sane maximum if you enforce one).
- Copy/read through user-pointer validation (no stray kernel dereference of malformed pointers).
- Emit to UART/console backend and return bytes written or a deterministic error.

4.8 Return Values and Errors (Phase Scope)

- **>= 0**: success (byte count where applicable).
- **< 0**: errors
  - **-1**: invalid syscall ID
  - **-2**: invalid descriptor or argument
  - **-3**: invalid user pointer or protection violation in syscall context

4.9 User-Space Helpers and Examples

Header and wrappers (user_syscalls.h)

```
#pragma once
#include <stdint.h>
#include <stddef.h>

enum {
  SYS_YIELD = 0,
  SYS_EXIT  = 1,
  SYS_WRITE = 2,
};

static inline int32_t syscall3(uint32_t id, uint32_t a1, uint32_t a2, uint32_t a3) {
  register uint32_t r0 asm("r0") = id;
  register uint32_t r1 asm("r1") = a1;
  register uint32_t r2 asm("r2") = a2;
  register uint32_t r3 asm("r3") = a3;

  asm volatile(
    "svc #0\n"
    : "+r"(r0)
```

```
      : "r"(r1), "r"(r2), "r"(r3)
      : "memory"
   );
   return (int32_t)r0;
}

static inline int32_t sys_yield(void) {
   return syscall3(SYS_YIELD, 0, 0, 0);
}

static inline void sys_exit(int32_t code) {
   (void)syscall3(SYS_EXIT, (uint32_t)code, 0, 0);
   while (1) { }
}

static inline int32_t sys_write(int32_t fd, const void *buf, size_t len) {
   return syscall3(SYS_WRITE, (uint32_t)fd, (uint32_t)buf, (uint32_t)len);
}
```

Example A — cooperative yield plus output

```
#include "user_syscalls.h"

int main(void) {
   const char *msg = "[A] tick\n";
   for (int i = 0; i < 20; i++) {
      int32_t n = sys_write(1, msg, 9);
      if (n < 0) {
         /* optional */
      }
      sys_yield();
   }
   sys_exit(0);
   return 0;
}
```

Example B — negative-path checks for write

```
#include "user_syscalls.h"

int main(void) {
   int32_t r1 = sys_write(9, "X\n", 2);
   int32_t r2 = sys_write(1, (void *)0xFFFFFFFF, 8);
   int32_t r3 = sys_write(1, "[B] ok\n", 7);

   if (r1 < 0 && r2 < 0 && r3 == 7)
      sys_exit(0);
   else
```

```
    sys_exit(1);
  return 0;
}
```

4.10 Traces

Emit the syscall trace lines from **Section 3.8** for every user syscall on test and demo runs.

4.11 Syscall Acceptance Checklist

- At least one task repeatedly executes **write + yield** without kernel instability.
- **exit** removes the task permanently from the runnable set (never resumed).
- Invalid **write** inputs return errors and **do not** corrupt kernel memory.
- Traces contain paired syscall entry/exit lines as specified in **Section 3.8** (rows **4** and **5**).

5) Faults, Memory Protection, and Isolation

5.1 Objective

Fault containment: invalid execution or access from user mode is classified; the offending task is terminated or quarantined per written policy; the **scheduler** and kernel stay alive **continuing runnable peers**.

5.2 Traces

Emit **Section 3.8** rows **6** (reason=fault) and **7** (reason=fault_recovery) whenever the exception path runs.

5.3 Flow — Fault Containment

- User causes invalid access or protection violation → CPU enters **data abort/prefetch abort** (or vendor-equivalent vector) handling.
- Handler **classifies**: invalid mapping, privilege violation, alignment error, permission fault, or categorized bucket you document consistently.
- Kernel marks offending process **terminated/blocked** and records reason for observability.
- Scheduler runs; next **healthy** user task resumes in USR via exception return.

Expected behavior: kernel alive; peers continue; failure reason observable in PCB/log state.

6) Process Control Block and Unified Context Expectations

The PCB/TCB must support Sections 3–5 in one coherent structure:

- **Privilege/mode**: saved CPSR snapshot and registers needed for a correct exception return back to USR.
- **Trap-frame anchor**: pointer or embedded save area for interrupted **USR** execution; ordering is shared across **IRQ**, **svc**, and **abort** return paths for this phase.
- **Syscall transient state**: return value destined for **r0**, syscall ID if logged.
- **Fault/termination**: fault type, termination reason, exit code linkage.
- **Scheduling**: **ready / running / waiting / terminated** (or your state names) consistent with Phase 1.

7) Deliverables

- Updated kernel source: user/kernel boundary, IRQ return to USR, SVC pipeline, fault handling per policy.
- PCB/TCB as in Section 6.
- **Syscall ABI** document (IDs, registers, return/error table, unknown-ID behavior).
- **Fault-handling policy** document (classification → outcome matrix).
- Demonstration run: ordinary preemption, successful syscalls from user tasks, one **faulting** user task **isolated** without full system collapse.
- Test report: pass/fail checklist and keyed traces.

8) Acceptance Criteria (Definition of Success)

- Timer-driven context switch remains stable for sustained runtime.
- At least **two** user-mode tasks concurrent under forced preemption.
- **yield** and **exit** correct; **write** validated (optional only if formally deferred in deviation log — default is validate).
- Faulting task can be terminated/isolated while kernel and peers continue.
- No user-to-kernel **privilege escalation** path observable in structured review or tests.
- No context corruption after repeated **IRQ + syscall + fault** interleavings.